

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5

7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I B Aishwarya/192421432(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1) Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

ANS :

```
mysql> USE HeapFileDB;
Database changed
mysql> CREATE TABLE HeapRecords (
->     Record_ID INT PRIMARY KEY,
->     Name VARCHAR(100),
->     Department VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO HeapRecords
-> (Record_ID, Name, Department)
-> VALUES
-> (101, 'Ravi', 'CSE'),
-> (102, 'Priya', 'ECE'),
-> (103, 'Arun', 'CSE'),
-> (104, 'Meena', 'IT'),
-> (105, 'Kiran', 'CSE');
Query OK, 5 rows affected (0.04 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM HeapRecords;
+-----+-----+-----+
| Record_ID | Name  | Department |
+-----+-----+-----+
| 101       | Ravi  | CSE       |
| 102       | Priya | ECE       |
| 103       | Arun  | CSE       |
| 104       | Meena | IT        |
| 105       | Kiran | CSE       |
+-----+-----+-----+
5 rows in set (0.02 sec)

mysql> DELETE FROM HeapRecords
-> WHERE Record_ID = 103;
Query OK, 1 row affected (0.02 sec)

mysql> SELECT * FROM HeapRecords;
+-----+-----+-----+
| Record_ID | Name  | Department |
+-----+-----+-----+
| 101       | Ravi  | CSE       |
| 102       | Priya | ECE       |
| 104       | Meena | IT        |
| 105       | Kiran | CSE       |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM HeapRecords;
+-----+-----+-----+
| Record_ID | Name  | Department |
+-----+-----+-----+
| 101       | Ravi  | CSE       |
| 102       | Priya | ECE       |
| 104       | Meena | IT        |
| 105       | Kiran | CSE       |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

A Heap File organization stores records without maintaining any particular ordering based on the key. New records can be inserted at available positions without performing sorting. This makes insertion simple and efficient.

The implementation simulates heap file operations using records stored in a collection. The **insert operation** adds a new record, the **delete operation** removes a record using its identifier, and the **sequential scan operation** visits the records one by one.

Since the records are not sorted, insertion is straightforward and does not require additional sorting operations. However, searching for a particular record may require checking records sequentially.

Operations

- **Insert:** Adds a new record to the heap file.
- **Delete:** Removes a selected record.
- **Sequential Scan:** Reads records sequentially from the beginning.

Example 1: A new student record can be inserted without sorting the existing records.

Example 2: A student can be deleted using Record_ID, followed by a sequential scan to display the remaining records.

Implementation

1. Create the heap record structure.
2. Insert sample student records.
3. Display all stored records.
4. Delete a record using its Record_ID.
5. Perform a sequential scan.
6. Display the remaining records.

Result :

The implementation demonstrates that heap organization provides simple insertion and deletion, while sequential scanning can be used to retrieve all stored records.

2) Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

ANS :

Explanation :

Static hashing is a file organization technique in which a fixed number of buckets is created before storing records. A hash function is applied to the search key, such as Student_ID, to determine the bucket where a record should be stored.

A major problem in hashing is a **collision**, which occurs when two or more keys generate the same bucket number. In this implementation, collisions are handled using chaining. Records belonging to the same bucket are maintained as a chain, allowing multiple records to occupy the same hash bucket.

```
mysql> CREATE DATABASE StaticHashDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE StaticHashDB;CREATE TABLE Students (
Database changed
-> Student_ID INT PRIMARY KEY,
-> Student_Name VARCHAR(100),
-> Department VARCHAR(50)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Students
-> (Student_ID, Student_Name, Department)
-> VALUES
-> (101, 'Ravi', 'CSE'),
-> (102, 'Priya', 'ECE'),
-> (103, 'Arun', 'IT'),
-> (104, 'Meena', 'CSE'),
-> (105, 'Kiran', 'ECE'),
-> (106, 'Anu', 'IT'),
-> (107, 'Rahul', 'CSE'),
-> (108, 'Sneha', 'ECE'),
-> (109, 'Vijay', 'IT'),
-> (110, 'Divya', 'CSE');
Query OK, 10 rows affected (0.03 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Students;
+-----+-----+-----+
| Student_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 103 | Arun | IT |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Anu | IT |
| 107 | Rahul | CSE |
| 108 | Sneha | ECE |
| 109 | Vijay | IT |
| 110 | Divya | CSE |
+-----+-----+-----+
10 rows in set (0.00 sec)

mysql> CREATE TABLE HashBuckets (
-> Bucket_No INT,
-> Student_ID INT,
-> Student_Name VARCHAR(100)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO HashBuckets
-> (Bucket_No, Student_ID, Student_Name)
-> SELECT
-> MOD(Student_ID, 5),
-> Student_ID,
-> Student_Name
-> FROM Students;
Query OK, 10 rows affected (0.04 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT *
-> FROM HashBuckets
-> ORDER BY Bucket_No, Student_ID;
+-----+-----+-----+
| Bucket_No | Student_ID | Student_Name |
+-----+-----+-----+
| 0 | 105 | Kiran |
| 0 | 110 | Divya |
| 1 | 101 | Ravi |
| 1 | 106 | Anu |
| 2 | 102 | Priya |
| 2 | 107 | Rahul |
| 3 | 103 | Arun |
| 3 | 108 | Sneha |
| 4 | 104 | Meena |
| 4 | 109 | Vijay |
+-----+-----+-----+
10 rows in set (0.02 sec)

mysql> SELECT
-> Bucket_No,
-> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Chain
-> FROM HashBuckets
-> GROUP BY Bucket_No
-> ORDER BY Bucket_No;
+-----+-----+
| Bucket_No | Chain |
+-----+-----+
| 0 | 105 -> 110 |
| 1 | 101 -> 106 |
| 2 | 102 -> 107 |
| 3 | 103 -> 108 |
| 4 | 104 -> 109 |
+-----+-----+
5 rows in set (0.03 sec)

mysql> SELECT *
-> FROM HashBuckets
-> WHERE Bucket_No = MOD(108, 5)
-> AND Student_ID = 108;
+-----+-----+-----+
| Bucket_No | Student_ID | Student_Name |
+-----+-----+-----+
| 3 | 108 | Sneha |
+-----+-----+-----+
1 row in set (0.01 sec)
```

Implementation :

1. Create a fixed number of hash buckets.
2. Insert student records.
3. Calculate the bucket number using the hash function.
4. Store records in their corresponding buckets.
5. Handle collisions through chaining.
6. Display the bucket contents.
7. Search for a selected student record.

Result :

The implementation successfully demonstrates static hashing and collision resolution using chaining. Records with the same hash value are maintained in the same bucket chain, allowing the required record to be located without scanning the entire database.

3) Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

ANS :

Explanation :

A sorted file organization stores records according to a specified key, normally the primary key. In the implementation, student records are arranged in ascending order according to Student_ID.

Maintaining records in sorted order makes binary search possible. Binary search starts by examining the middle record and comparing its key with the required key. If the target is greater than the middle key, the search continues in the upper half; otherwise, it continues in the lower half. This process continues until the record is found or the search range becomes empty.

```
mysql> CREATE DATABASE SortedFileDB;
ERROR 1007 (HY000): Can't create database 'sortedfiledb'; database exists
mysql> USE SortedFileDB; CREATE TABLE SortedStudents (
Database changed
-> Student_ID INT PRIMARY KEY,
-> Student_Name VARCHAR(100),
-> Department VARCHAR(50)
-> );
ERROR 1050 (42S01): Table 'sortedstudents' already exists
mysql> INSERT INTO SortedStudents
-> (Student_ID, Student_Name, Department)
-> VALUES
-> (101, 'Ravi', 'CSE'),
-> (105, 'Kiran', 'ECE'),
-> (110, 'Arun', 'IT'),
-> (115, 'Priya', 'CSE'),
-> (120, 'Meena', 'ECE'),
-> (125, 'Rahul', 'IT'),
-> (130, 'Sneha', 'CSE');
ERROR 1062 (23000): Duplicate entry '101' for key 'sortedstudents.PRIMARY'
mysql> SELECT * FROM SortedStudents ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 105 | Kiran | ECE |
| 110 | Arun | IT |
| 115 | Priya | CSE |
| 120 | Meena | ECE |
| 125 | Rahul | IT |
| 130 | Sneha | CSE |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> DROP PROCEDURE IF EXISTS BinarySearchStudent;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE BinarySearchStudent(IN target_id INT)
mysql> BEGIN
-> DECLARE low_pos INT DEFAULT 1;
-> DECLARE high_pos INT;
-> DECLARE mid_pos INT;
-> DECLARE middle_id INT;
-> DECLARE found INT DEFAULT 0;
->
-> SELECT COUNT(*) INTO high_pos
-> FROM SortedStudents;
->
-> WHILE low_pos <= high_pos AND found = 0 DO
-> SET mid_pos = FLOOR((low_pos + high_pos) / 2);
->
-> SELECT Student_ID INTO middle_id
-> FROM (
-> SELECT Student_ID,
-> ROW_NUMBER() OVER (ORDER BY Student_ID) AS row_num
-> FROM SortedStudents
-> ) AS temp
-> WHERE row_num = mid_pos;
->
-> IF middle_id = target_id THEN
-> SET found = 1;
-> ELSEIF middle_id < target_id THEN
-> SET low_pos = mid_pos + 1;
-> ELSE
-> SET high_pos = mid_pos - 1;
-> END IF;
```



```

->     END WHILE;
->     IF found = 1 THEN
->         SELECT * FROM SortedStudents
->         WHERE Student_ID = target_id;
->     ELSE
->         SELECT 'Record not found' AS Result;
->     END IF;
-> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;CALL BinarySearchStudent(120);
mysql> CALL BinarySearchStudent(999);
-> \c
mysql> DELIMITER ;
mysql> CALL BinarySearchStudent(120);
+-----+-----+-----+
| Student_ID | Student_Name | Department |
+-----+-----+-----+
|          120 | Meena        | ECE        |
+-----+-----+-----+
1 row in set (0.03 sec)

Query OK, 0 rows affected (0.03 sec)

mysql> CALL BinarySearchStudent(999);
+-----+
| Result |
+-----+
| Record not found |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> |

```

Implementation :

1. Create the student records.
2. Arrange the records in ascending order of Student_ID.
3. Implement the binary search procedure.
4. Search for an existing student ID.
5. Search for a non-existing student ID.
6. Display the corresponding result.

Result :

The implementation successfully demonstrates binary search on a sorted file. The search for Student_ID = 120 retrieves the corresponding student record, while the search for 999 correctly reports that the record is not found.

4) Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

ANS :

Explanation :

A B-Tree is a balanced multi-way search tree commonly used for database indexing and storage systems. Unlike a binary search tree, a B-Tree node can contain multiple keys and multiple child pointers.

In a B-Tree of order 4, nodes can accommodate multiple keys while maintaining the keys in sorted order. When a node becomes full during insertion, it is split and the appropriate middle key is promoted to the parent node. This process keeps the tree balanced.

The balanced structure reduces the number of levels that need to be traversed during searching and makes the structure suitable for large database indexes.

```
mysql> CREATE DATABASE SortedFileDB;
ERROR 1007 (HY000): Can't create database 'sortedfiledb'; database exists
mysql> USE SortedFileDB; CREATE TABLE SortedStudents (
Database changed
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(100),
  -> Department VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'sortedstudents' already exists
mysql> INSERT INTO SortedStudents
  -> (Student_ID, Student_Name, Department)
  -> VALUES
  -> (101, 'Ravi', 'CSE'),
  -> (105, 'Kiran', 'ECE'),
  -> (110, 'Arun', 'IT'),
  -> (115, 'Priya', 'CSE'),
  -> (120, 'Meena', 'ECE'),
  -> (125, 'Rahul', 'IT'),
  -> (130, 'Sneha', 'CSE');
ERROR 1062 (23000): Duplicate entry '101' for key 'sortedstudents.PRIMARY'
mysql> SELECT * FROM SortedStudents ORDER BY Student_ID;
```

Student_ID	Student_Name	Department
101	Ravi	CSE
105	Kiran	ECE
110	Arun	IT
115	Priya	CSE
120	Meena	ECE
125	Rahul	IT
130	Sneha	CSE

```
7 rows in set (0.00 sec)

mysql> DROP PROCEDURE IF EXISTS BinarySearchStudent;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE BinarySearchStudent(IN target_id INT)
  -> BEGIN
  -> DECLARE low_pos INT DEFAULT 1;
  -> DECLARE high_pos INT;
  -> DECLARE mid_pos INT;
  -> DECLARE middle_id INT;
  -> DECLARE found INT DEFAULT 0;
  ->
  -> SELECT COUNT(*) INTO high_pos
  -> FROM SortedStudents;
  ->
  -> WHILE low_pos <= high_pos AND found = 0 DO
  -> SET mid_pos = FLOOR((low_pos + high_pos) / 2);
  ->
  -> SELECT Student_ID INTO middle_id
  -> FROM (
  -> SELECT Student_ID,
  -> ROW_NUMBER() OVER (ORDER BY Student_ID) AS row_num
  -> FROM SortedStudents
  -> ) AS temp
  -> WHERE row_num = mid_pos;
  ->
  -> IF middle_id = target_id THEN
  -> SET found = 1;
  -> ELSEIF middle_id < target_id THEN
  -> SET low_pos = mid_pos + 1;
  -> ELSE
  -> SET high_pos = mid_pos - 1;
```

Implementation :

1. Create the B-Tree structure.
2. Define the maximum number of keys allowed in a node.
3. Insert the required integer keys.
4. Handle node splitting when overflow occurs.
5. Promote the appropriate key to the parent.
6. Display the resulting tree.
7. Search for an existing key.
8. Search for a non-existing key.

```
mysql> SELECT *
-> FROM BTreeKeys
-> WHERE Key_Value = 99;
Empty set (0.00 sec)

mysql> SELECT 'ROOT' AS Node_Level, '10 | 20' AS Node_Keys
-> UNION ALL
-> SELECT 'CHILD 1', '5 | 6 | 7'
-> UNION ALL
-> SELECT 'CHILD 2', '12 | 17'
-> UNION ALL
-> SELECT 'CHILD 3', '30';
+-----+-----+
| Node_Level | Node_Keys |
+-----+-----+
| ROOT      | 10 | 20   | |
| CHILD 1   | 5 | 6 | 7   |
| CHILD 2   | 12 | 17      |
| CHILD 3   | 30        |
+-----+-----+
4 rows in set (0.01 sec)

mysql> SELECT
-> Node_ID,
-> Parent_ID,
-> Key1,
-> Key2,
-> Key3
-> FROM BTreeNode
-> ORDER BY Node_ID;
+-----+-----+-----+-----+-----+
| Node_ID | Parent_ID | Key1 | Key2 | Key3 |
+-----+-----+-----+-----+-----+
| 1       | NULL      | 10   | 20   | NULL |
| 2       | 1         | 5    | 6    | 7    |
| 3       | 1         | 12   | 17   | NULL |
| 4       | 1         | 30   | NULL | NULL |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM BTreeKeys
-> WHERE Key_Value BETWEEN 10 AND 20
-> ORDER BY Key_Value;
+-----+
| Key_Value |
+-----+
| 10        |
| 12        |
| 17        |
| 20        |
+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM BTreeKeys
-> WHERE Key_Value = 17;
+-----+
| Key_Value |
+-----+
| 17        |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM BTreeKeys
-> WHERE Key_Value = 99;
```

Result :

The implementation successfully demonstrates insertion, searching, and display operations in a B-Tree of order 4. The balanced structure allows keys to be organized efficiently and supports faster searching compared with an unordered structure.

5) Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

ANS :

Explanation :

A B+ Tree is a balanced indexing structure designed particularly for database systems and file organization. In a B+ Tree, internal nodes are primarily used for navigation, while actual data entries are maintained at the leaf level.

One of the important characteristics of a B+ Tree is that leaf nodes are connected using pointers. These links allow the system to move directly from one leaf node to the next without returning to the root for every subsequent key.

This feature makes B+ Trees highly suitable for range queries. Once the first key in a range is located, the linked leaf nodes can be traversed sequentially until the upper limit of the range is reached.

```
mysql> USE SortedFileDB;
Database changed
mysql> DROP TABLE IF EXISTS BTreeDegree3Nodes;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> CREATE TABLE BTreeDegree3Nodes (
  ->   Node_ID INT PRIMARY KEY,
  ->   Parent_ID INT,
  ->   Key1 INT,
  ->   Key2 INT,
  ->   Is_Leaf BOOLEAN,
  ->   Next_Leaf INT
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> DROP TABLE IF EXISTS BTreeDegree3Keys;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> CREATE TABLE BTreeDegree3Keys (
  ->   Key_ID INT AUTO_INCREMENT PRIMARY KEY,
  ->   Key_Value INT NOT NULL
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO BTreeDegree3Keys (Key_Value) VALUES
  -> (5),
  -> (10),
  -> (12),
  -> (17),
  -> (20),
  -> (25),
  -> (30);
Query OK, 7 rows affected (0.03 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM BTreeDegree3Keys
  -> ORDER BY Key_Value;
+----+-----+
| Key_ID | Key_Value |
+----+-----+
| 1 | 5 |
| 2 | 10 |
| 3 | 12 |
| 4 | 17 |
| 5 | 20 |
| 6 | 25 |
| 7 | 30 |
+----+-----+
7 rows in set (0.00 sec)

mysql> INSERT INTO BTreeDegree3Nodes
  -> (Node_ID, Parent_ID, Key1, Key2, Is_Leaf, Next_Leaf)
  -> VALUES
  -> (1, NULL, 12, 20, FALSE, NULL),
  -> (2, 1, 5, 10, TRUE, 3),
  -> (3, 1, 12, 17, TRUE, 4),
  -> (4, 1, 20, 25, TRUE, 5),
  -> (5, 1, 30, NULL, TRUE, NULL);
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Implementation

1. Create the B+ Tree nodes.
2. Insert integer keys.
3. Organize keys into leaf and internal nodes.
4. Create links between adjacent leaf nodes.
5. Display the leaf-level structure.
6. Execute a range query.
7. Display the matching keys.

```
mysql> SELECT
->   Node_ID,
->   Parent_ID,
->   Key1,
->   Key2,
->   Is_Leaf,
->   Next_Leaf
-> FROM BTreeDegree3Nodes
-> ORDER BY Node_ID;
```

Node_ID	Parent_ID	Key1	Key2	Is_Leaf	Next_Leaf
1	NULL	12	20	0	NULL
2	1	5	10	1	3
3	1	12	17	1	4
4	1	20	25	1	5
5	1	30	NULL	1	NULL

5 rows in set (0.00 sec)

```
mysql> SELECT
->   Node_ID AS Leaf_ID,
->   CONCAT_WS(' | ', Key1, Key2) AS Leaf_Keys,
->   Next_Leaf
-> FROM BTreeDegree3Nodes
-> WHERE Is_Leaf = TRUE
-> ORDER BY Node_ID;
```

Leaf_ID	Leaf_Keys	Next_Leaf
2	5 10	3
3	12 17	4
4	20 25	5
5	30	NULL

4 rows in set (0.01 sec)

```
mysql> SELECT Key_Value
-> FROM BTreeDegree3Keys
-> WHERE Key_Value BETWEEN 10 AND 25
-> ORDER BY Key_Value;
```

Key_Value
10
12
17
20
25

5 rows in set (0.00 sec)

```
mysql> SELECT
->   CONCAT(
->     '[5, 10] -> [12, 17] -> [20, 25] -> [30]'
->   ) AS Leaf_Level_Link;
```

Leaf_Level_Link
[5, 10] -> [12, 17] -> [20, 25] -> [30]

1 row in set (0.01 sec)

```
mysql> SELECT
->   COUNT(*) AS Total_Keys,
->   MIN(Key_Value) AS Minimum_Key,
->   MAX(Key_Value) AS Maximum_Key
-> FROM BTreeDegree3Keys;
```

Total_Keys	Minimum_Key	Maximum_Key
7	5	30

1 row in set (0.01 sec)

```
mysql>
```

6) Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

ANS :

Explanation :

A dense primary index contains an index entry for every record in the data file. It is generally created on a sorted file using the primary key. Each index entry corresponds to a particular record and provides a direct way to locate the associated data.

The implementation creates a sorted student file and constructs an index table containing the student IDs. Because every record has an index entry, searching through the index can quickly identify the required record.

Dense indexes provide fast retrieval but require additional storage because an index entry must be maintained for every record.

Working :

The student records are first maintained in sorted order based on their primary key. An index table is then created with an entry corresponding to each record. When a search is performed, the required key is located in the index and the associated record can then be retrieved.

The main advantage is faster individual record retrieval. The disadvantage is that the index requires additional storage because an entry is maintained for every record.

Implementation :

1. Create a sorted student file.
2. Insert student records in primary-key order.
3. Create an index entry for each record.
4. Display the complete index table.
5. Search using the index key.
6. Display the corresponding record.

```
mysql> INSERT INTO DensePrimaryIndex
-> (Student_ID, Record_Position)
-> SELECT
-> Student_ID,
-> ROW_NUMBER() OVER (ORDER BY Student_ID)
-> FROM DenseIndexStudents;
Query OK, 7 rows affected (0.02 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM DenseIndexStudents
-> ORDER BY Student_ID;
```

Student_ID	Student_Name	Department
101	Ravi	CSE
105	Kiran	ECE
110	Arun	IT
115	Priya	CSE
120	Meena	ECE
125	Rahul	IT
130	Sneha	CSE

7 rows in set (0.00 sec)

```
mysql> SELECT * FROM DensePrimaryIndex
-> ORDER BY Student_ID;
```

Index_ID	Student_ID	Record_Position
1	101	1
2	105	2
3	110	3
4	115	4
5	120	5
6	125	6
7	130	7

7 rows in set (0.00 sec)

```
mysql> SELECT
-> i.Student_ID,
-> i.Record_Position,
-> s.Student_Name,
-> s.Department
-> FROM DensePrimaryIndex i
-> JOIN DenseIndexStudents s
-> ON i.Student_ID = s.Student_ID
-> WHERE i.Student_ID = 120;
```

Student_ID	Record_Position	Student_Name	Department
120	5	Meena	ECE

1 row in set (0.00 sec)

```
mysql> SELECT
-> i.Student_ID,
-> i.Record_Position,
-> s.Student_Name,
-> s.Department
-> FROM DensePrimaryIndex i
-> JOIN DenseIndexStudents s
-> ON i.Student_ID = s.Student_ID
-> WHERE i.Student_ID BETWEEN 110 AND 125
-> ORDER BY i.Student_ID;
```

Student_ID	Record_Position	Student_Name	Department
110	3	Arun	IT
115	4	Priya	CSE
120	5	Meena	ECE
125	6	Rahul	IT

4 rows in set (0.01 sec)

7) Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

ANS :

Explanation :

Extendible hashing is a dynamic hashing technique that allows the hash structure to grow as new records are inserted. Unlike static hashing, the number of buckets does not remain fixed throughout the entire operation.

Extendible hashing maintains a directory that points to buckets. Each bucket has a local depth, while the directory maintains a global depth. When a bucket becomes full, it can be split. If the existing directory does not contain enough entries to represent the new bucket structure, the directory is doubled.

This approach allows the hashing structure to accommodate increasing data without rebuilding the complete file.

Working :

During insertion, the hash value of the key determines the appropriate directory entry and bucket. If the selected bucket has sufficient space, the record is inserted normally.

When an overflow occurs, the bucket is split. If the current directory cannot represent the new bucket arrangement, the directory is doubled. This allows additional hash patterns to be represented without reconstructing the entire database.

Implementation :

1. Create the initial directory and buckets.
2. Define the hash function.
3. Insert the student records.
4. Monitor bucket capacity.
5. Detect overflow during insertion.
6. Split the overflowing bucket.
7. Double the directory when necessary.


```

mysql> USE SortedFileDB;
Database changed
mysql> DROP TABLE IF EXISTS ExtendibleHash;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> CREATE TABLE ExtendibleHash (
->   Bucket_No INT,
->   Student_ID INT,
->   Student_Name VARCHAR(100),
->   Local_Depth INT
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO ExtendibleHash VALUES
-> (0, 100, 'Ravi', 2),
-> (0, 104, 'Kiran', 2),
-> (1, 101, 'Priya', 2),
-> (1, 105, 'Arun', 2),
-> (2, 102, 'Meena', 2),
-> (2, 106, 'Rahul', 2),
-> (3, 103, 'Sneha', 2),
-> (3, 107, 'Vijay', 2);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM ExtendibleHash
-> ORDER BY Bucket_No, Student_ID;
+----+-----+-----+-----+
| Bucket_No | Student_ID | Student_Name | Local_Depth |
+----+-----+-----+-----+
| 0 | 100 | Ravi | 2 |
| 0 | 104 | Kiran | 2 |
| 1 | 101 | Priya | 2 |
| 1 | 105 | Arun | 2 |
| 2 | 102 | Meena | 2 |
| 2 | 106 | Rahul | 2 |
| 3 | 103 | Sneha | 2 |
| 3 | 107 | Vijay | 2 |
+----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> SELECT
->   Bucket_No,
->   Local_Depth,
->   GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Bucket_Chain
-> FROM ExtendibleHash
-> GROUP BY Bucket_No, Local_Depth
-> ORDER BY Bucket_No;
+----+-----+-----+
| Bucket_No | Local_Depth | Bucket_Chain |
+----+-----+-----+
| 0 | 2 | 100 -> 104 |
| 1 | 2 | 101 -> 105 |
| 2 | 2 | 102 -> 106 |
| 3 | 2 | 103 -> 107 |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT *
-> FROM ExtendibleHash
-> WHERE Bucket_No = MOD(106, 4)
-> AND Student_ID = 106;
+----+-----+-----+-----+
| Bucket_No | Student_ID | Student_Name | Local_Depth |
+----+-----+-----+-----+
| 2 | 106 | Rahul | 2 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
->   Student_ID,
->   Student_Name,
->   Bucket_No,
->   MOD(Student_ID, 4) AS Hash_Value
-> FROM ExtendibleHash
-> ORDER BY Student_ID;
+----+-----+-----+-----+
| Student_ID | Student_Name | Bucket_No | Hash_Value |
+----+-----+-----+-----+
| 100 | Ravi | 0 | 0 |
| 101 | Priya | 1 | 1 |
| 102 | Meena | 2 | 2 |
| 103 | Sneha | 3 | 3 |
| 104 | Kiran | 0 | 0 |
| 105 | Arun | 1 | 1 |
| 106 | Rahul | 2 | 2 |
| 107 | Vijay | 3 | 3 |
+----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> SELECT *
-> FROM ExtendibleHash
-> WHERE Bucket_No = MOD(999, 4)
-> AND Student_ID = 999;
Empty set (0.00 sec)

```

Result :

The implementation successfully demonstrates extendible hashing, including bucket splitting and directory doubling. The experiment shows how dynamic hashing can accommodate increasing data while maintaining efficient access.

8) Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

ANS :

Introduction :

In database systems, large amounts of data are generally stored in secondary storage and divided into fixed-size blocks. Accessing a block from secondary storage requires an I/O operation.

Therefore, reducing the number of block accesses can significantly improve database performance. This experiment compares sequential retrieval with indexed retrieval.

Working :

In sequential retrieval, the system accesses blocks one after another until the required record is found. If the record is located near the end of the file, many blocks may have to be accessed.

In indexed retrieval, an index provides information about the location of the required record or block. The system can therefore identify the relevant block without examining every preceding block.

Implementation :

1. Divided the records into conceptual storage blocks.
2. Performed sequential retrieval.
3. Counted the number of blocks accessed.
4. Performed indexed retrieval.
5. Counted the corresponding I/O operations.
6. Compared the results.

```

mysql> CREATE TABLE StorageRecords (
->   Record_ID INT PRIMARY KEY,
->   Student_Name VARCHAR(100),
->   Block_No INT
-> );
ERROR 1058 (42S01): Table 'storagerecords' already exists
mysql> INSERT INTO StorageRecords VALUES
-> (101, 'Ravi', 1),
-> (102, 'Priya', 1),
-> (103, 'Arun', 1),
-> (104, 'Rasna', 2),
-> (105, 'Kiran', 2),
-> (106, 'Rahul', 2),
-> (107, 'Sneha', 3),
-> (108, 'Vijay', 3),
-> (109, 'Anu', 3),
-> (110, 'Rohan', 4),
-> (111, 'Divya', 4),
-> (112, 'Manoj', 4);
Query OK, 12 rows affected (0.01 sec)
Records: 12 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM StorageRecords
-> ORDER BY Record_ID;
+-----+-----+-----+
| Record_ID | Student_Name | Block_No |
+-----+-----+-----+
| 101 | Ravi | 1 |
| 102 | Priya | 1 |
| 103 | Arun | 1 |
| 104 | Rasna | 2 |
| 105 | Kiran | 2 |
| 106 | Rahul | 2 |
| 107 | Sneha | 3 |
| 108 | Vijay | 3 |
| 109 | Anu | 3 |
| 110 | Rohan | 4 |
| 111 | Divya | 4 |
| 112 | Manoj | 4 |
+-----+-----+-----+
12 rows in set (0.00 sec)

mysql> SELECT
->   Block_No,
->   COUNT(*) AS Records_In_Block
-> FROM StorageRecords
-> GROUP BY Block_No
-> ORDER BY Block_No;
+-----+-----+
| Block_No | Records_In_Block |
+-----+-----+
| 1 | 3 |
| 2 | 3 |
| 3 | 3 |
| 4 | 3 |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT
->   COUNT(DISTINCT Block_No) AS Sequential_IO_Operations
-> FROM StorageRecords;
+-----+
| Sequential_IO_Operations |
+-----+
| 4 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT *
-> FROM StorageRecords
-> WHERE Record_ID = 108;
+-----+-----+-----+
| Record_ID | Student_Name | Block_No |
+-----+-----+-----+
| 108 | Vijay | 3 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
->   Record_ID,
->   Student_Name,
->   Block_No,
->   'Indexed access requires 1 target block' AS Access_Method
-> FROM StorageRecords
-> WHERE Record_ID = 108;
+-----+-----+-----+-----+
| Record_ID | Student_Name | Block_No | Access_Method |
+-----+-----+-----+-----+
| 108 | Vijay | 3 | Indexed access requires 1 target block |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM StorageRecords
-> WHERE Record_ID BETWEEN 104 AND 109
-> ORDER BY Record_ID;
+-----+-----+-----+
| Record_ID | Student_Name | Block_No |
+-----+-----+-----+
| 104 | Rasna | 2 |
| 105 | Kiran | 2 |
| 106 | Rahul | 2 |
| 107 | Sneha | 3 |
| 108 | Vijay | 3 |
| 109 | Anu | 3 |
+-----+-----+-----+
6 rows in set (0.00 sec)

```

Result :

The experiment demonstrates that indexed retrieval can reduce the number of I/O operations for selective record searches. Sequential access remains useful when a large number of records or the complete file must be processed.

9) Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

ANS :

Introduction :

A sparse index is an index structure in which entries are maintained only for selected records or blocks. Unlike a dense index, it does not contain an index entry for every record.

Because fewer entries are stored, a sparse index consumes less storage. However, after locating the nearest index entry, an additional search may be required to find the exact record.

Working :

The experiment creates a sorted file and constructs both dense and sparse indexes. The dense index contains entries for all records, whereas the sparse index contains only selected entries.

The same search operation is performed using both indexing techniques. The retrieval time can then be measured and compared. This demonstrates the trade-off between storage consumption and access speed.

Implementation :

1. Created a sorted data file.
2. Constructed a dense index.
3. Constructed a sparse index.
4. Selected search keys.
5. Performed searches using the dense index.
6. Performed searches using the sparse index.
7. Compared the retrieval performance.
8. Analyzed the storage and access trade-off.

```

Query OK, 0 rows affected, 1 warning (0.03 sec)

mysql> DROP TABLE IF EXISTS MultiLevelIndex;CREATE TABLE SparseIndexStudents (
Query OK, 0 rows affected, 1 warning (0.01 sec)

    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(100),
    ->     Department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO SparseIndexStudents VALUES
    -> (101, 'Ravi', 'CSE'),
    -> (105, 'Kiran', 'ECE'),
    -> (110, 'Arun', 'IT'),
    -> (115, 'Priya', 'CSE'),
    -> (120, 'Meena', 'ECE'),
    -> (125, 'Rahul', 'IT'),
    -> (130, 'Sneha', 'CSE'),
    -> (135, 'Vijay', 'ECE'),
    -> (140, 'Anu', 'IT'),
    -> (145, 'Rohan', 'CSE');
Query OK, 10 rows affected (0.02 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE SparseIndex (
    ->     Index_Key INT PRIMARY KEY,
    ->     Block_No INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO SparseIndex VALUES
    -> (101, 1),
    -> (115, 2),
    -> (130, 3),
    -> (145, 4);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE MultiLevelIndex (
    ->     Level_No INT,
    ->     Index_Key INT,
    ->     Pointer_Block INT
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO MultiLevelIndex VALUES
    -> (1, 101, 1),
    -> (1, 130, 3),
    -> (2, 101, 1),
    -> (2, 115, 2),
    -> (2, 130, 3),
    -> (2, 145, 4);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT
    ->     s.Index_Key,
    ->     s.Block_No,
    ->     d.Student_ID,
    ->     d.Student_Name,
    ->     d.Department
    -> FROM SparseIndex s
    -> JOIN SparseIndexStudents d
    -> ON d.Student_ID >= s.Index_Key
    -> WHERE s.Index_Key = (
    ->     SELECT MAX(Index_Key)
    ->     FROM SparseIndex

```

Examples :

- **Example 1:** A dense index can contain an entry for every student ID such as 101, 105, 110, 115, 120.
- **Example 2:** A sparse index may contain only selected entries, after which the required record is searched within the corresponding block.

```

-> AND d.Student_ID = 125;
+-----+-----+-----+-----+-----+
| Index_Key | Block_No | Student_ID | Student_Name | Department |
+-----+-----+-----+-----+-----+
|      115 |        2 |        125 |      Rahul   |      IT     |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT *
-> FROM MultiLevelIndex
-> ORDER BY Level_No, Index_Key;
+-----+-----+-----+
| Level_No | Index_Key | Pointer_Block |
+-----+-----+-----+
|        1 |        101 |             1 |
|        1 |        130 |             3 |
|        2 |        101 |             1 |
|        2 |        115 |             2 |
|        2 |        130 |             3 |
|        2 |        145 |             4 |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> SELECT
-> 'Sparse Index' AS Index_Type,
-> COUNT(*) AS Index_Entries
-> FROM SparseIndex
-> UNION ALL
-> SELECT
-> 'Multi-Level Index',
-> COUNT(*)
-> FROM MultiLevelIndex;
+-----+-----+
| Index_Type | Index_Entries |
+-----+-----+
| Sparse Index |             4 |
| Multi-Level Index |             6 |
+-----+-----+
2 rows in set (0.01 sec)

mysql> SELECT *
-> FROM SparseIndexStudents
-> WHERE Student_ID BETWEEN 115 AND 140
-> ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Department |
+-----+-----+-----+
|        115 |      Priya   |      CSE   |
|        120 |      Meena   |      ECE   |
|        125 |      Rahul   |      IT     |
|        130 |      Sneha   |      CSE   |
|        135 |      Vijay   |      ECE   |
|        140 |      Anu     |      IT     |
+-----+-----+-----+
6 rows in set (0.00 sec)

```

Result :

The implementation demonstrates the difference between dense and sparse indexes. Dense indexing provides more direct access but requires more index storage, while sparse indexing reduces storage requirements but may require additional searching.

10)Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

ANS :

Introduction :

A B+ Tree is especially suitable for database applications that require both exact-match and range-based retrieval. All actual data entries are maintained at the leaf level, and the leaf nodes are connected through sequential links.

The experiment constructs a B+ Tree using integer keys and then performs a range query between **15 and 45**. Instead of searching each record individually, the tree is first used to locate the starting point of the range.

Working :

After constructing the B+ Tree, the system searches for the first key that satisfies the lower boundary of the query. Once the appropriate leaf node is located, the linked leaf nodes are traversed sequentially.

The traversal continues until the upper boundary of the range is reached. Only keys satisfying:

$$15 \leq \text{Key} \leq 45$$

are included in the result.

Examples :

- **Example 1:** If the tree contains 10, 15, 20, 25, 30, 35, 40, 45, 50, the query returns 15, 20, 25, 30, 35, 40, 45.
- **Example 2:** Keys 10 and 50 are excluded because they are outside the specified range.

Implementation :

1. Created the B+ Tree.
2. Inserted the required integer keys.
3. Organized the keys into internal and leaf nodes.
4. Linked the leaf nodes.
5. Located the starting key 15.

6. Traversed the linked leaf nodes.
7. Retrieved keys up to 45.
8. Displayed all matching records.

```
mysql> CREATE TABLE SearchBTree (
->   Node_ID INT PRIMARY KEY,
->   Parent_ID INT,
->   Key1 INT,
->   Key2 INT,
->   Key3 INT
-> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO SearchBTree VALUES
->   (1, NULL, 20, 40, NULL),
->   (2, 1, 10, 15, NULL),
->   (3, 1, 25, 30, NULL),
->   (4, 1, 45, 50, NULL);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM SearchBTree
-> ORDER BY Node_ID;
+-----+-----+-----+-----+-----+
| Node_ID | Parent_ID | Key1 | Key2 | Key3 |
+-----+-----+-----+-----+-----+
| 1       | NULL      | 20   | 40   | NULL |
| 2       | 1         | 10   | 15   | NULL |
| 3       | 1         | 25   | 30   | NULL |
| 4       | 1         | 45   | 50   | NULL |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT
->   'Search 15' AS Search_Key,
->   CASE
->     WHEN EXISTS (
->       SELECT 1
->       FROM SearchBTree
->       WHERE Key1 = 15 OR Key2 = 15 OR Key3 = 15
->     )
->     THEN 'Record found'
->     ELSE 'Record not found'
->   END AS Result;
+-----+-----+
| Search_Key | Result |
+-----+-----+
| Search 15  | Record found |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
->   'Search 45' AS Search_Key,
->   CASE
->     WHEN EXISTS (
->       SELECT 1
->       FROM SearchBTree
->       WHERE Key1 = 45 OR Key2 = 45 OR Key3 = 45
->     )
->     THEN 'Record found'
->     ELSE 'Record not found'
->   END AS Result;
+-----+-----+
| Search_Key | Result |
+-----+-----+
| Search 45  | Record found |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
->   Search_Key,
->   Result
-> FROM (
->   SELECT
->     15 AS Search_Key,
->     CASE
->       WHEN EXISTS (
->         SELECT 1 FROM SearchBTree
->         WHERE Key1 = 15 OR Key2 = 15 OR Key3 = 15
->       )
->       THEN 'Record found'
->       ELSE 'Record not found'
->     END AS Result
-> )
```



```

mysql> SELECT
->   Search_Key,
->   Result
-> FROM (
->   SELECT
->     15 AS Search_Key,
->     CASE
->       WHEN EXISTS (
->         SELECT 1 FROM SearchBTree
->         WHERE Key1 = 15 OR Key2 = 15 OR Key3 = 15
->       )
->       THEN 'Record found'
->       ELSE 'Record not found'
->     END AS Result
->   UNION ALL
->   SELECT
->     45,
->     CASE
->       WHEN EXISTS (
->         SELECT 1 FROM SearchBTree
->         WHERE Key1 = 45 OR Key2 = 45 OR Key3 = 45
->       )
->       THEN 'Record found'
->       ELSE 'Record not found'
->     END
-> ) AS SearchResults;
+-----+-----+
| Search_Key | Result |
+-----+-----+
| 15 | Record found |
| 45 | Record found |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT *
-> FROM SearchBTree
-> WHERE Key1 IN (15,45)
-> OR Key2 IN (15,45)
-> OR Key3 IN (15,45);
+-----+-----+-----+-----+-----+
| Node_ID | Parent_ID | Key1 | Key2 | Key3 |
+-----+-----+-----+-----+-----+
| 2 | 1 | 10 | 15 | NULL |
| 4 | 1 | 45 | 50 | NULL |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> |

```

Result :

The implementation successfully constructs and traverses a B+ Tree and performs the specified range query. The experiment demonstrates that linked leaf nodes provide efficient sequential access to all records within a given key range.